

Работа с устройствами в Linux «с нуля»

Владислав Богданов

2026-06-16

Оглавление

1. Базовая философия: «Всё есть файл»	1
2. Шаг за шагом: от простого к надёжному	1
3. Собираем всё вместе: анатомия идеального цикла чтения	3
4. Главные правила, которые нужно запомнить	4
5. Как двигаться дальше?	4
6. Базовый пример (его нужно дописать)	5
7. Исправления от ИИ	9

Цель: Написать надежную программу на C, которая общается с устройством (например, /dev/radio0), не зависает и корректно обрабатывает все возможные ситуации.

1. Базовая философия: «Всё есть файл»

В Linux нет специального набора команд для работы с «железом». Для операционной системы ваше устройство (радиомодуль, датчик, мотор) выглядит как обычный файл в папке /dev/.

Это значит, что мы используем те же 4 базовые операции, что и для работы с текстовым файлом:

1. Открыть (open)
2. Прочитать (read) или Записать (write)
3. Управлять спец. настройками (ioctl – пока опустим)
4. Закрыть (close)



Аналогия: Представьте, что устройство – это почтовый ящик. Чтобы положить или забрать письмо, вы должны сначала открыть ящик ключом (*open*), затем работать с ним (*read/write*), а в конце закрыть (*close*), чтобы не занимать ресурсы.

2. Шаг за шагом: от простого к надежному

Шаг 1. Открытие устройства

```
int fd = open("/dev/radio0", O_RDWR | O_NONBLOCK);
```

- fd (file descriptor) – это просто число (например, 3 или 4). Это ваш «талон» или «пульт» для работы с этим конкретным открытым файлом.
- O_RDWR – открываем и для чтения (Read), и для записи (Write).
- O_NONBLOCK – критически важный флаг. Он говорит системе: «Если данных прямо сейчас нет, не вешай мою программу, а сразу верни управление, чтобы я мог заняться другими делами». Без этого флага программа может «заморозиться» навечно, ожидая ответа от устройства.

Шаг 2. Проблема наивного чтения (и как её решить)

Новички часто пишут так:

```
// ПЛОХОЙ ПРИМЕР
char buffer[100];
read(fd, buffer, 100); // Ожидаем, что прочитаем ровно 100 байт
```

Почему это плохо? Драйвер устройства не обязан отдавать все 100 байт за один раз. Он может отдать 10 байт сейчас, 30 через миллисекунду и 60 позже. Это называется **частичное чтение (partial read)**.

Правильный подход: Читать в цикле, пока не наберем всё, что нужно, или пока не поймем, что сообщение закончилось (например, пришел символ `\n`).

Шаг 3. Почему нельзя просто крутить `read` в цикле?

Если мы напишем `while(1) { read(...); }`, программа будет бесконечно спрашивать: «Есть данные? Нет? А сейчас? Нет?». Это называется **активное ожидание (busy-waiting)**. Процессор будет загружен на 100%, нагреваться и тратить энергию впустую.

Решение: Использовать системный вызов `poll`.



Аналогия: Вместо того чтобы стоять у почтового ящика и проверять его каждую секунду, вы нанимаете вахтера (`poll`). Вы говорите ему: «Позвони мне, когда придет письмо, но не жди дольше 2 секунд». Пока писем нет, ваша программа «спит» и не грузит процессор.

```
struct pollfd pfd;
pfd.fd = fd;           // Наш файловый дескриптор
pfd.events = POLLIN;   // Нас интересует событие "можно читать" (данные пришли)

// Ждем максимум 2000 мс (2 секунды)
int result = poll(&pfd, 1, 2000);

if (result == 0) {
    // Прошло 2 секунды, данных нет (таймаут)
} else if (result > 0) {
    // Данные есть! Можно безопасно вызывать read(), он не заблокирует программу
}
```

Шаг 4. Проблема наивной записи

С записью (`write`) та же история, что и с чтением. Вы просите систему отправить 100 байт, но внутренний буфер устройства может быть заполнен, и система согласится принять только 40 байт.

Правильный подход: Цикл записи со смещением.

```
size_t total_written = 0;
while (total_written < total_len) {
    // Пишем только оставшуюся часть данных
    ssize_t bytes = write(fd, data + total_written, total_len - total_written);

    if (bytes > 0) {
        total_written += bytes; // Успех, сдвигаем счетчик
    } else if (errno == EAGAIN) {
```

```

        // Буфер устройства переполнен. Система говорит: "Попробуй позже"
        // Делаем крошечную паузу и повторяем попытку
        usleep(10000);
    }
}

```

3. Собираем всё вместе: анатомия идеального цикла чтения

Теперь, когда мы поняли почему код выглядит именно так, давайте посмотрим на логику чтения из вашего примера с комментариями для новичка:

```

#define MAX_MSG_LEN 1024
char full_buffer[MAX_MSG_LEN];
size_t total_bytes = 0;

struct pollfd pfd = { .fd = fd, .events = POLLIN };

// Читаем, пока не заполним буфер (оставляя 1 байт под символ '\0')
while (total_bytes < MAX_MSG_LEN - 1) {

    // 1. Спрашиваем у "вахтера": есть данные? Ждем не более 2 сек.
    int poll_ret = poll(&pfd, 1, 2000);

    if (poll_ret == 0) {
        printf("Таймаут: устройство молчит.\n");
        break; // Выходим из цикла, чтобы не висеть вечно
    }
    if (poll_ret < 0) {
        perror("Ошибка poll");
        break;
    }

    // 2. Вахтер сказал "ДА", данные точно есть. Читаем их.
    // Важно: читаем в КОНЕЦ уже накопленных данных (full_buffer + total_bytes)
    ssize_t bytes = read(fd, full_buffer + total_bytes, MAX_MSG_LEN - 1 -
total_bytes);

    if (bytes > 0) {
        total_bytes += bytes; // Увеличиваем счетчик прочитанного

        // 3. Проверяем, не конец ли это сообщения (например, пришел перенос строки)
        if (full_buffer[total_bytes - 1] == '\n' || full_buffer[total_bytes - 1] ==
'\0') {
            break; // Сообщение полное, выходим из цикла досрочно
        }
    } else if (bytes == 0) {
        printf("Устройство закрыло соединение (EOF).\n");
    }
}

```

```

        break;
    } else {
        // Ошибка чтения. Если это EAGAIN, просто продолжаем цикл (данных пока нет)
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            continue;
        }
        perror("Критическая ошибка чтения");
        break;
    }
}

// 4. Гарантированно делаем строку валидной (завершаем нулем)
full_buffer[total_bytes] = '\0';
printf("Получено: %s\n", full_buffer);

```

4. Главные правила, которые нужно запомнить

- Всегда проверяйте возвращаемое значение. `open`, `read`, `write` и `poll` могут завершиться неудачей.
- Всегда проверяйте, что результат ≥ 0 (или > 0 для `poll`).
- Никогда не верьте, что `read` или `write` отработают за один раз. Пишите циклы с накоплением (`total_bytes += bytes`).
- Используйте `O_NONBLOCK + poll`. Это золотой стандарт для создания отзывчивых программ, которые не «вешают» систему, ожидая медленное устройство.
- Всегда закрывайте дескриптор. `close(fd)` в конце программы обязателен. Незакрытые дескрипторы приводят к утечкам ресурсов (утечкам памяти на уровне ядра).

5. Как двигаться дальше?

Теперь, когда базовая механика понятна, следующие шаги для улучшения этого кода (когда вы будете готовы):

- Вынести логику в функции. Сделать `int read_device(int fd, char *buf, size_t max_len)`, чтобы `main` оставался чистым и читаемым.
- Заменить `usleep` на `poll` с `POLLOUT`. Для записи это более профессиональный подход, чем пауза в цикле.
- Добавить обработку `EINTR`. Если программу прервет внешний сигнал (например, пользователь нажмет `Ctrl+C` в другом потоке), системный вызов может прерваться досрочно. Хороший код просто повторяет вызов в этом случае.

6. Базовый пример (его нужно дописать)

Нужны доработки:

Что нужно исправить для релиза:

- . Модульность (Разделение ответственности):

Весь код в `main` – это антипаттерн для продакшена. Логику чтения и записи нужно вынести в отдельные функции. Это облегчит тестирование, повторное использование и чтение кода (что соответствует вашему предпочтению структурировать логику).

- . Замена устаревшего `usleep`:

Функция `usleep` удалена из стандарта POSIX.1-2008. Её нужно заменить на `nanosleep` или, что еще профессиональнее, на `poll` с флагом `POLLOUT` (ожидание возможности записи).

- . Устранение "магических чисел":

Числа 2000, 1024, 10000 должны быть именованными константами (`#define` или `enum`) в начале файла.

- . Логика определения режима:

Проверка `if(strlen(msg)>0)` ненадежна. Если пользователь передаст пустую строку "" третьим аргументом, логика сломается. Надежнее опираться на `argc >= 3`.

- . Коды возврата:

Возврат `-1` из `main` не является стандартом. Следует использовать `EXIT_FAILURE (1)` и `EXIT_SUCCESS (0)` из `<stdlib.h>`.

- . Помощь пользователю (Usage):

Релизная утилита должна выводить справку, если аргументы переданы неверно или запрошен `--help`.

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h> // atoi
#include <poll.h>

#define MODE_READ 0
#define MODE_WRITE 1

int main(int argc, char *argv[]){
    const char *dev_path_radio0 = "/dev/radio0";
    const char *dev_path_radio1 = "/dev/radio1";
    const char *dev_path_radio2 = "/dev/radio2";

    int fd;
    char buffer[8];
    int device_num = 0; // default radio0

    if (argc >= 2) {
```

```

    device_num = atoi(argv[1]);
    if(device_num < 0 || device_num > 2){
        fprintf(stderr, "Invalid device number. Use 0, 1, or 2\n");
        return -1;
    }
    printf("Select device: %d\n", device_num);

}
const char *msg = "\0";
if (argc >= 3) {
    msg = argv[2];
    printf("msg : %s\n", msg);
}

// select device
const char *dev_path;
switch(device_num) {
    case 0: dev_path = dev_path_radio0; break;
    case 1: dev_path = dev_path_radio1; break;
    case 2: dev_path = dev_path_radio2; break;
    default: dev_path = dev_path_radio0;
}
printf("Path: %s\n", dev_path);

//select current mode
int mode = MODE_READ;
if(strlen(msg)>0) mode = MODE_WRITE;
const char *mode_str = (mode==MODE_WRITE) ? "write" : "read";
printf("Current mode: %s\n", mode_str);

// open device
fd = open(dev_path, O_RDWR | O_NONBLOCK);
if (fd < 0) {
    fprintf(stderr, "No open %s: %s\n", dev_path, strerror(errno));
    return -1;
}
printf("Device open - OK (fd = %d)\n", fd);

// run operation
if(mode==MODE_READ){
    //Создаем большой буфер для накопления полного сообщения
    #define MAX_MSG_LEN 1024
    char full_buffer[MAX_MSG_LEN];
    size_t total_bytes = 0;

    //Настраиваем структуру для poll

    struct pollfd pfd;
    pfd.fd = fd;

```



```

pfd.events = POLLIN;

printf("Ожидание данных от устройства...\n");

while(total_bytes < MAX_MSG_LEN - 1) {
    // Ждем появления данных максимум 2000 мс (2 секунды)
    // Если данных нет, poll вернет 0, и мы не будем грузить CPU

    int poll_ret = poll(&pfd, 1, 2000);

    if(poll_ret < 0) {
        perror("Ошибка poll");
        break;
    } else if(poll_ret == 0) {
        printf("Таймаут: устройство не ответило за 2 секунды.\n");
        break;
    }

    // Данные есть! Читаем их в конец нашего буфера
    // Оставляем место для завершающего '\0'

    ssize_t bytes = read(fd, full_buffer + total_bytes, MAX_MSG_LEN -
1 - total_bytes);
    if(bytes > 0) {
        total_bytes += bytes;

        // ОПЦИОНАЛЬНО: Если твой драйвер завершает сообщения
        переносом строки,
        // мы можем прервать цикл досрочно, как только получили '\n'
        if (full_buffer[total_bytes - 1] == '\n' || full_buffer[total_bytes -
1] == '\0')
        {
            break;
        }
        } else if(bytes == 0) {
            printf("Устройство вернуло EOF (конец потока).\n");
            break;
        } else {
            // При использовании poll() ошибка EAGAIN маловероятна,
            // но на всякий случай обрабатываем её корректно (просто продолжаем
цикл)

            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                continue;
            }
            perror("Ошибка чтения");
            break;
        }
    }
}

```

```

    }

    full_buffer[total_bytes] = '\0';
    printf("=====\n");
    printf("Полное сообщение получено (%zu байт):\n%s\n", total_bytes,
full_buffer);
    printf("=====\n");

} else {
// operation WRITE
    const char *data_to_send = msg;
    size_t total_len = strlen(data_to_send);
    size_t total_written = 0;

    printf("Send data %zu bytes ... \n", total_len);

    while(total_written < total_len) {
        // Пишем начиная с текущего смещения, оставшееся количество байт
        ssize_t bytes = write(fd, data_to_send + total_written, total_len -
total_written);

        if(bytes < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK) {
                // В неблокирующем режиме буфер устройства переполнен.
                // Можно сделать небольшую паузу или использовать poll() с POLLOUT
                usleep(10000); // 10 мс
                continue;
            } else if (errno == EINTR){
                continue; // Прервано сигналом, пробуем снова
            } else {
                perror("Критическая ошибка записи");
                break;
            }
        } else if(bytes == 0) {
            // Запись 0 байт обычно означает ошибку или закрытие, но формально это
не прогресс
            fprintf(stderr, "Запись 0 байт. Прерываем.\n");
            break;
        }

        // Успешно записали 'bytes' байт. Сдвигаем указатель и счетчик
        total_written += bytes;
        printf("Записано: %zd из %zu байт\n", total_written, total_len);
    }

    if(total_written == total_len) {
        printf("Успешно отправлено всё сообщение.\n");
    }

}

```

```
close(fd);
printf("Device close\n");
}
```

7. Исправления от ИИ

Почему эти изменения делают код "релизным":

1. `strtol` вместо `atoi`: `atoi` опасен, так как не сообщает об ошибках (если передать "abc", он вернет 0, и программа молча откроет `/dev/radio0`). `strtol` позволяет надежно валидировать ввод.
2. Массив указателей `dev_paths`: Убрал `switch`, заменив его на безопасный доступ по индексу массива. Это сокращает код и снижает вероятность ошибки копипасты.
3. Функции `read_from_device` и `write_to_device`: `main` теперь читается как оглавление. Если завтра вам нужно будет добавить логирование в файл или отправку данных по сети, вы измените только одну маленькую функцию, не трогая остальное.
4. `nanosleep` вместо `usleep`: Это современный стандарт POSIX. Мы также вынесли задержку в константу `WRITE_RETRY_DELAY_MS`, чтобы её можно было легко тюнить.
5. Безопасный вывод: В `printf` для буфера используется `%.s` с явным указанием длины. Это 100% защита от вывода мусора, если по какой-то причине `\0` не встанет на место (хотя в коде он гарантирован).
6. Обработка `EINTR` в `poll`: Добавлена. Если ОС пошлет сигнал (например, фоновый таймер или `SIGCHLD`), `poll` прервется с ошибкой `EINTR`. Правильное поведение – не падать, а просто вызвать `poll` снова.
7. `sigaction` вместо `signal`: `signal()` имеет разное поведение в разных версиях UNIX/Linux (особенно в отношении сброса обработчика после срабатывания и `SA_RESTART`). `sigaction` ведет себя предсказуемо везде.
8. Отсутствие `SA_RESTART`: Мы намеренно не ставим этот флаг. Если `poll` или `read` прерваны сигналом, они вернут `-1` и `errno = EINTR`. Это позволяет циклу мгновенно заметить изменение `keep_running` и выйти, а не ждать окончания таймаута в 2 секунды.
9. `volatile sig_atomic_t`: Это единственный стандартный способ в C передать информацию из контекста прерывания (сигнала) в основной поток выполнения без гонки данных (data race).
10. Чистота обработчика: В `signal_handler` нет ничего, кроме присваивания 0. Никаких `printf("Exiting...\n")`, которые могут привести к неопределенному поведению (undefined behavior), если сигнал пришел в момент, когда основной поток уже держал внутренний мьютекс `stdio`.
11. Гарантированная очистка: Независимо от того, где именно был нажат `Ctrl+C`, управление дойдет до строки `close(fd)`, что удовлетворяет вашим требованиям к надежному управлению ресурсами.

```
#include <fcntl.h>
```

```

#include <stdio.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>
#include <poll.h>
#include <time.h>
#include <signal.h>

// =====
// Конфигурация
// =====
#define MAX_DEVICES      3
#define MAX_MSG_LEN      1024
#define READ_TIMEOUT_MS  2000
#define WRITE_RETRY_DELAY_MS 10

#define MODE_READ        0
#define MODE_WRITE       1

// =====
// Глобальные переменные
// =====
// volatile гарантирует, что компилятор не закэширует значение в регистре.
// sig_atomic_t гарантирует атомарность чтения/записи (безопасно из обработчика
сигнала).
static volatile sig_atomic_t keep_running = 1;

// =====
// Прототипы функций
// =====
static void signal_handler(int signum);
static void print_usage(const char *prog_name);
static int read_from_device(int fd, char *buffer, size_t max_len);
static int write_to_device(int fd, const char *data, size_t len);

// =====
// Главная функция
// =====
int main(int argc, char *argv[]) {
    // 1. Настройка обработки сигналов (SIGINT = Ctrl+C, SIGTERM = kill)
    struct sigaction sa;
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0; // Намеренно НЕ используем SA_RESTART, чтобы системные вызовы
                    // возвращали EINTR и мы могли быстро проверить keep_running

    if (sigaction(SIGINT, &sa, NULL) == -1 || sigaction(SIGTERM, &sa, NULL) == -1) {
        perror("Не удалось настроить обработчик сигналов");
        return EXIT_FAILURE;
    }
}

```

```

// 2. Обработка запроса помощи
if (argc > 1 && (strcmp(argv[1], "-h") == 0 || strcmp(argv[1], "--help") == 0)) {
    print_usage(argv[0]);
    return EXIT_SUCCESS;
}

// 3. Парсинг номера устройства (безопасная замена atoi)
int device_num = 0;
if (argc >= 2) {
    char *endptr;
    long val = strtol(argv[1], &endptr, 10);
    if (*endptr != '\0' || val < 0 || val >= MAX_DEVICES) {
        fprintf(stderr, "Ошибка: Неверный номер устройства. Используйте 0, 1 или
2.\n");
        return EXIT_FAILURE;
    }
    device_num = (int)val;
}

// 4. Определение режима и данных
// Режим записи включается только если явно передан 3-й аргумент
int mode = (argc >= 3) ? MODE_WRITE : MODE_READ;
const char *msg = (mode == MODE_WRITE) ? argv[2] : "";

// 5. Выбор пути к устройству
const char *dev_paths[MAX_DEVICES] = {
    "/dev/radio0",
    "/dev/radio1",
    "/dev/radio2"
};
const char *dev_path = dev_paths[device_num];

printf("Устройство: %s | Режим: %s\n", dev_path, (mode == MODE_WRITE) ? "Запись" :
"Чтение");

// 6. Открытие устройства
int fd = open(dev_path, O_RDWR | O_NONBLOCK);
if (fd < 0) {
    fprintf(stderr, "Ошибка открытия %s: %s\n", dev_path, strerror(errno));
    return EXIT_FAILURE;
}
printf("Устройство успешно открыто (fd = %d)\n", fd);

// 7. Выполнение операции
int result = EXIT_SUCCESS;
if (mode == MODE_READ) {
    char full_buffer[MAX_MSG_LEN];
    if (read_from_device(fd, full_buffer, sizeof(full_buffer)) < 0) {
        result = EXIT_FAILURE;
    }
}

```

```

    } else {
        if (write_to_device(fd, msg, strlen(msg)) < 0) {
            result = EXIT_FAILURE;
        }
    }
}

// 8. Корректное завершение и очистка ресурсов
if (!keep_running) {
    printf("\n[Получен сигнал завершения. Очистка ресурсов...]\n");
}

close(fd); // Гарантированно закрываем дескриптор при любом сценарии
printf("Устройство закрыто корректно.\n");

return result;
}

// =====
// Реализация вспомогательных функций
// =====

static void print_usage(const char *prog_name) {
    printf("Использование: %s [номер_устройства] [сообщение]\n", prog_name);
    printf("  номер_устройства : 0, 1 или 2 (по умолчанию 0)\n");
    printf("  сообщение       : если указано, программа перейдет в режим записи\n");
    printf("\nПримеры:\n");
    printf("  %s 1           # Читать из /dev/radio1\n", prog_name);
    printf("  %s 2 \"HELLO\"    # Записать \"HELLO\" в /dev/radio2\n", prog_name);
}

static void signal_handler(int signum) {
    (void)signum; // Подавляем предупреждение о неиспользуемом параметре
    keep_running = 0; // Минимально возможное действие для асинхронной безопасности
}

static int read_from_device(int fd, char *buffer, size_t max_len) {
    size_t total_bytes = 0;
    struct pollfd pfd = { .fd = fd, .events = POLLIN };

    printf("Ожидание данных (таймаут %d мс)...\n", READ_TIMEOUT_MS);

    // Проверка keep_running в условии цикла для быстрого выхода
    while (total_bytes < max_len - 1 && keep_running) {
        int poll_ret = poll(&pfd, 1, READ_TIMEOUT_MS);

        if (!keep_running) break; // Немедленный выход, если пришел сигнал

        if (poll_ret < 0) {
            if (errno == EINTR) continue; // Прервано сигналом, повторяем poll
            perror("Ошибка poll");
            return -1;
        }
    }
}

```

```

    }
    if (poll_ret == 0) {
        printf("Таймаут: устройство не ответило.\n");
        break;
    }

    // Данные есть, читаем
    ssize_t bytes = read(fd, buffer + total_bytes, max_len - 1 - total_bytes);

    if (bytes > 0) {
        total_bytes += bytes;
        // Проверка на конец сообщения (драйверы часто используют \n)
        if (buffer[total_bytes - 1] == '\n' || buffer[total_bytes - 1] == '\0') {
            break;
        }
    } else if (bytes == 0) {
        printf("Устройство вернуло EOF.\n");
        break;
    } else {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            continue; // Ложное срабатывание poll, безопасный повтор
        }
        perror("Ошибка чтения");
        return -1;
    }
}

// Гарантируем нуль-терминированную строку
buffer[total_bytes] = '\0';

// Выводим результат только если мы не были прерваны (чтобы не засорять консоль
при Ctrl+C)
if (keep_running || total_bytes > 0) {
    printf("=====\n");
    printf("Получено (%zu байт):\n%.*s", total_bytes, (int)total_bytes, buffer);
    // Добавляем перенос строки, если сообщение не заканчивалось на него
    if (total_bytes > 0 && buffer[total_bytes - 1] != '\n') {
        printf("\n");
    }
    printf("=====\n");
}

return keep_running ? 0 : -1;
}

static int write_to_device(int fd, const char *data, size_t len) {
    if (len == 0) {
        printf("Пустое сообщение, запись пропущена.\n");
        return 0;
    }
}

```

```

size_t total_written = 0;
printf("Отправка %zu байт...\n", len);

// Структура для nanosleep (современная замена устаревшему usleep)
struct timespec retry_delay = { 0, WRITE_RETRY_DELAY_MS * 1000000L };

// Проверка keep_running в условии цикла
while (total_written < len && keep_running) {
    ssize_t bytes = write(fd, data + total_written, len - total_written);

    if (!keep_running) break; // Немедленный выход при сигнале

    if (bytes < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            // Буфер драйвера переполнен. Ждем и пробуем снова.
            nanosleep(&retry_delay, NULL);
            continue;
        } else if (errno == EINTR) {
            continue; // Прервано сигналом, повторяем write
        } else {
            perror("Критическая ошибка записи");
            return -1;
        }
    } else if (bytes == 0) {
        fprintf(stderr, "Запись 0 байт. Неожиданное поведение драйвера.\n");
        return -1;
    }

    total_written += bytes;
}

if (keep_running) {
    printf("Успешно отправлено %zu байт.\n", total_written);
    return 0;
} else {
    printf("\n[Запись прервана пользователем после %zu байт]\n", total_written);
    return -1;
}
}

```